

RODESENCE

Frontend <-> Backend Integration Specification

Version 1.0 | Hackathon build contract

Based on the official PARAKRAM 1.0 problem statement PK01PS001 and the team's current architecture decisions.

CORE PRINCIPLE: The backend is the contract. Flutter must depend only on documented API behavior, not database tables, internal service names, or AI implementation details.

Technology target: Flutter + Fastify 5 + PostgreSQL/PostGIS + AWS S3/SQS + separate ML worker

1. What We Are Building

The official challenge is centered on proactive road-surface inspection from dashcam footage: extract frames, detect multiple road-damage forms with confidence and bounding boxes, classify severity, synchronize detections with positional data, visualize the results geographically, and provide repair prioritization or maintenance routing. The citizen reporting and worker-management ideas remain useful as an operational extension, but they are not the primary technical challenge.

Layer	Primary responsibility	Owner
Flutter frontend	Capture inputs, show processing state, render maps/results, role-based screens, send API requests	Frontend
Fastify API	Authentication, authorization, API contract, orchestration, metadata, queries, signed URLs, jobs	Backend
PostgreSQL + PostGIS	Durable data, geospatial queries, detection/route metadata	Backend/DB
S3	Raw videos, extracted frames, result artifacts	AWS/Backend
SQS	Asynchronous processing jobs	AWS/Backend
ML worker	Frame extraction, preprocessing, YOLO/equivalent detection, severity inputs	AI/ML
Map UI	Markers, filters, details, route/cluster visualization	Frontend
Priority engine	Severity + road/traffic context -> repair priority	Backend/Analytics

2. End-to-End System Flow

Flutter -> Fastify -> create processing job -> S3 video -> SQS -> ML worker -> PostgreSQL/PostGIS -> Fastify -> Flutter map/dashboard

The HTTP request must never wait for heavy video processing or YOLO inference. The client receives a job identifier and observes the processing state. This is the key boundary between the product API and the ML pipeline.

3. Shared Vocabulary

Term	Meaning	Frontend uses	Backend uses
Video	An uploaded dashcam recording or provided road footage	Upload state, duration, processing status	S3 object, capture metadata, processing job
Frame	A sampled image derived from a video	Optional processing preview	Timestamp, frame index, quality flags
Detection	One model-detected road-damage instance	Marker/detail card	Bounding box, type, confidence, severity, location
Severity	Consistent classification/score for damage significance	Badges/charts/filter	Persisted score + class + factors
Position record	GPS/time record associated	Optional route context	Frame-to-GPS synchronization

	with capture		
Processing job	Long-running pipeline state	Progress indicator	Queue payload, retries, final status
Road issue	Operational/maintenance case derived from one or more detections or a manual report	Issue details/status	Deduplication, assignment, lifecycle
Priority	Decision-support score for repair order	Dashboard ranking	Calculated from documented factors

4. Backend Module Boundaries

Module	Responsibilities	Must NOT do
auth	Login, token validation, role checks, account state	Perform business logic for detections
videos	Video records, presigned upload URLs, metadata, job creation	Process video bytes inside request handler
frames	Frame metadata, quality flags, previews	Run YOLO in API process
detections	Detection CRUD/query, bounding boxes, confidence, categories	Invent AI results
severity	Severity calculation/versioning and factors	Hide the scoring methodology
geospatial	Nearby queries, bounding-box map queries, GPS synchronization metadata	Perform heavy image inference
jobs	Create/read processing jobs and states	Expose queue credentials to Flutter
prioritization	Priority score and explainable factors	Replace raw severity data
issues	Operational case aggregation, assignment/status lifecycle	Treat every frame as a new issue
admin	Dashboards, filters, configuration, worker management	Bypass authorization
audit	Security/critical action history	Store passwords or secrets

5. Roles and Authorization Contract

Role	Core permissions
END_USER	Upload manual report; view own reports; view permitted public detections/issues; receive status updates
WORKER	View assigned work; accept assignment; submit work progress/evidence; mark work complete
ADMIN	View authority scope; review detections/issues; assign workers; inspect priority; verify resolution
SUPER_ADMIN	System-wide administration; manage admins/configuration; inspect audit and processing health

Important: role alone is not sufficient. Every protected resource must also pass object/scope authorization.
 Example: an ADMIN may only access issues/detections inside the authority scope assigned to that account.

6. API Contract - Versioning and Conventions

Base URL: /api/v1

JSON: application/json

Dates: ISO-8601 UTC strings

Coordinates: WGS84 decimal degrees

IDs: opaque strings/UUIDs; frontend must treat IDs as opaque

Errors: { "success": false, "error": { "code": "...", "message": "...", "requestId": "..." } }

Frontend rule: never construct URLs from database IDs or internal S3 keys beyond documented API fields. Never depend on field order. Treat unknown fields as ignorable and required fields as contractually stable.

7. Authentication Endpoints

Method	Endpoint	Purpose	Auth
POST	/api/v1/auth/register	Create an account	Public
POST	/api/v1/auth/login	Authenticate and return access/refresh credentials	Public
POST	/api/v1/auth/refresh	Rotate/refresh access credentials	Refresh credential
GET	/api/v1/auth/me	Return current user and roles/scope	Authenticated
POST	/api/v1/auth/logout	Invalidate refresh session if implemented	Authenticated

POST /api/v1/auth/login

Request:

```
{ "email": "user@example.com", "password": "..." }
```

Response:

```
{
  "success": true,
  "data": {
    "accessToken": "...",
    "refreshToken": "...",
    "expiresIn": 900,
    "user": { "id": "...", "role": "END_USER", "scope": { "jurisdictionId": null } }
  }
}
```

8. Video Ingestion API

Method	Endpoint	Purpose	Response highlights
POST	/api/v1/videos/upload-url	Request a presigned upload URL	uploadUrl, objectKey, expiresAt
POST	/api/v1/videos	Create video metadata after upload initialization	videoId, status
GET	/api/v1/videos/:videoId	Read video metadata + processing state	video, processing
POST	/api/v1/videos/:videoId/process	Start/queue processing	jobId, status
GET	/api/v1/jobs/:jobId	Poll long-running job	status, progress, counts, error
POST	/api/v1/videos/:videoId/position-dataset	Attach positional data reference	positionDatasetId

POST /api/v1/videos/upload-url

```
{
  "filename": "route-01.mp4",
  "contentType": "video/mp4",
  "sizeBytes": 481239102
}
```

Response

```
{
  "success": true,
  "data": {
    "videoId": "vid_123",
    "uploadUrl": "https://...",
    "objectKey": "videos/vid_123/source.mp4",
    "expiresAt": "2026-08-26T09:00:00Z"
  }
}
```

Frontend upload sequence: request URL -> upload directly to S3 -> notify backend that upload is complete -> request processing. Do not send the video bytes through Fastify unless the team later has a specific reason.

9. Processing State Machine

UPLOAD_PENDING -> UPLOADED -> QUEUED -> EXTRACTING_FRAMES -> DETECTING -> SEVERITY_ANALYSIS -> GEO_SYNC -> COMPLETED

Terminal/error states: FAILED, CANCELLED

The frontend must be able to handle any state without assuming a fixed processing duration. Polling is acceptable for the MVP; WebSocket/SSE updates can be added later if needed.

Status	Frontend behavior
UPLOADED	Show upload complete; processing not started
QUEUED	Show waiting/queued state
EXTRACTING_FRAMES	Show frame extraction progress if available
DETECTING	Show AI detection in progress
SEVERITY_ANALYSIS	Show detection complete, analysis in progress

GEO_SYNC	Show location synchronization
COMPLETED	Enable map/results/dashboard
FAILED	Show recoverable error + retry action if backend allows
CANCELLED	Show cancelled state; do not continue polling indefinitely

10. Detection API

Method	Endpoint	Purpose
GET	/api/v1/detections/:detectionId	Detection details
GET	/api/v1/detections	Paginated list with filters
GET	/api/v1/detections/nearby	Nearby detections
GET	/api/v1/detections/geojson	Map-ready GeoJSON collection
GET	/api/v1/videos/:videoId/detections	Detections for one video

Detection object

```
{
  "id": "det_123",
  "videoId": "vid_123",
  "frameId": "frm_456",
  "damageType": "pothole",
  "confidence": 0.93,
  "boundingBox": { "x": 412, "y": 188, "width": 140, "height": 96 },
  "severity": { "score": 8.1, "class": "SEVERE", "version": "severity-v1" },
  "location": { "lat": 20.2961, "lng": 85.8245 },
  "captureTimestamp": "2026-08-26T06:15:17Z"
}
```

Frontend must render the bounding box only when frame dimensions are known and the response includes sufficient metadata. The map should use location, severity, category, and capture time - not internal database fields.

11. Map and Geospatial API

Method	Endpoint	Purpose
GET	/api/v1/map/detections	Map viewport query
GET	/api/v1/map/detections/geojson	Map-ready GeoJSON
GET	/api/v1/map/summary	Counts by severity/category/time
GET	/api/v1/detections/nearby	Radius query
GET	/api/v1/routes/:routeId	Detected route and maintenance context

GET /api/v1/map/detections?bbox=85.80,20.27,85.85,20.32&severity=SEVERE,HIGH&category=pothole,crack&from=2026-08-01&to=2026-08-26&page=1&limit=100

Recommendation: GeoJSON is the cleanest map contract for Flutter map layers if the chosen map SDK supports it. Otherwise return a documented marker DTO. The API should support viewport/bounding-box filtering to avoid downloading thousands of points at once.

12. Severity and Priority Contracts

Concept	Backend	Frontend
Severity score	Compute/persist score + class + methodology version	Show class, score, and explanation/factors when available
Priority score	Combine severity with contextual factors such as road category/traffic density	Rank issues and show why an issue is prioritized
Confidence	Persist raw model confidence	Display confidence only with appropriate wording; do not call it ground-truth accuracy
Ambiguous case	Return explicit uncertainty/borderline state	Do not force a fake binary decision

Priority response

```
{
  "score": 87,
  "tier": "CRITICAL",
  "factors": [
    { "name": "severity", "value": 8.9, "weight": 0.40 },
    { "name": "trafficDensity", "value": 9.1, "weight": 0.25 },
    { "name": "roadCategory", "value": 8.0, "weight": 0.20 },
    { "name": "confidence", "value": 9.4, "weight": 0.15 }
  ],
  "modelVersion": "priority-v1"
}
```

The official challenge explicitly asks for repair-prioritization based on severity plus contextual factors or, alternatively, optimized crew routing. The backend should therefore keep the scoring method versioned and explainable.

13. Manual Citizen Report Extension

This feature is optional relative to the official dashcam challenge, but it can make the solution feel like a complete civic platform. Model it as another source rather than redefining the whole system.

sourceType: DASHCAM | CITIZEN | INSPECTOR

Endpoint	Purpose
POST /api/v1/issues	Create manual issue/report
GET /api/v1/issues/:issueId	View operational issue
PATCH /api/v1/issues/:issueId/status	Authorized workflow change
POST /api/v1/issues/:issueId/assign	Assign worker
POST /api/v1/issues/:issueId/evidence	Upload repair evidence

14. What Flutter Must Implement

Frontend area	Responsibilities	API dependency
Auth screens	Login/register/session restore/role-based navigation	auth/*
Video ingestion	Choose/capture video, upload progress, retry/cancel UI	videos/upload-url, videos, jobs
Processing screen	Show current job state and progress	jobs/:jobId
Map	Render detections, filters, selection cards, clustering/viewport loading	map/*, detections/*
Detection detail	Frame/image preview, box, type, confidence, severity, timestamp, location	detections/:id
Admin dashboard	Counts, priorities, filters, assignments	admin/map/priority APIs
Worker workflow	Assignments, status, evidence upload	issues/assign/status
User report	Manual issue creation and status tracking	issues/*
Error/empty states	Handle expired URL, failed job, no data, permission denied	All APIs

15. Frontend Non-Negotiables

- Never store AWS access keys in Flutter.
- Never call SQS, RDS, or the ML worker directly from Flutter.
- Never rely on database structure or S3 object naming beyond API responses.
- Always send the access token through the documented authorization mechanism.
- Always handle 401, 403, 404, 409, 422, 429, and 5xx responses.
- Debounce map/filter requests and cancel stale requests where the Flutter networking layer supports it.
- Paginate or viewport-limit large detection lists; do not request the entire city map in one call.
- Treat processing as asynchronous and persistent across app restarts.
- Use server timestamps for authoritative event time; do not invent status times locally.
- Keep UI state keyed by stable IDs and not by list position.

16. Backend Non-Negotiables

- Validate every request at the API boundary.
- Enforce authorization on every protected resource, not only at route group level.
- Use Argon2id for password hashing if local authentication is retained.
- Rate-limit authentication and expensive endpoints.
- Keep S3 private; use short-lived presigned URLs.
- Never perform heavy video decoding or YOLO inference inside an HTTP request handler.
- Use idempotent job creation/processing semantics so retries do not duplicate detections.
- Persist model/version metadata with AI results.
- Keep raw detection, severity, and priority data separate and traceable.
- Use PostGIS for spatial queries rather than application-side distance calculations where possible.
- Use structured logs and request IDs; never log passwords, tokens, or raw secrets.

17. Error Contract

```
{
  "success": false,
  "error": {
    "code": "PROCESSING_JOB_FAILED",
    "message": "Video processing failed. Please retry.",
    "requestId": "req_abc123",
    "details": null
  }
}
```

HTTP	Meaning	Flutter action
400	Malformed request	Fix client input; do not retry blindly
401	Missing/expired authentication	Refresh session or send user to login
403	Authenticated but unauthorized	Show permission message
404	Resource not found	Show not-found state
409	State/conflict/idempotency problem	Refresh resource and retry only if appropriate
422	Validation failure	Highlight fields / request data
429	Rate limited	Back off and retry later
500/502/503	Server/dependency failure	Show transient error; retry where safe

18. Data Model - First Version

Entity	Key fields
users	id, email, passwordHash, role, jurisdictionId, status, createdAt
videos	id, storageKey, sourceType, captureStart, captureEnd, status, createdBy
frames	id, videoId, frameIndex, timestamp, storageKey, width, height, qualityStatus
position_records	id, videoId, timestamp, latitude, longitude, speed, heading
detections	id, frameId, damageType, confidence, bbox, severityId, latitude, longitude, captureTimestamp
severity_results	id, detectionId, score, class, factorsjson, version
processing_jobs	id, videoId, type, status, progress, attempts, error, createdAt, completedAt
issues	id, sourceType, linkedDetectionId/clusterId, reporterId, status, priority, createdAt
assignments	id, issueId, workerId, status, assignedAt, completedAt
audit_logs	id, actorId, action, entityType, entityId, metadata, createdAt

The exact schema may change after the team decides the positional-data format and ML result format. The API contract should remain stable even if the persistence schema evolves.

19. ML Worker Contract

The ML developer should NOT need to know Fastify route internals. The worker consumes a documented job payload and emits documented result records.

Job input

```
{
  "jobId": "job_123",
  "videoId": "vid_123",
  "sourceKey": "videos/vid_123/source.mp4",
  "framePolicy": { "intervalMs": 500 },
  "positionDatasetId": "pos_123"
}
```

Detection result

```
{
  "frameIndex": 42,
  "frameTimestamp": "2026-08-26T06:15:17Z",
  "damageType": "crack",
  "confidence": 0.91,
  "bbox": { "x": 120, "y": 80, "width": 240, "height": 110 }
}
```

The ML worker can be implemented in Python with FFmpeg/OpenCV and YOLO/equivalent detection. Node/Fastify should orchestrate, store, query, and expose results. The challenge explicitly requires frame extraction/preprocessing and an object detection architecture with bounding boxes and confidence scores.

20. AWS Ownership Boundary

Service	Backend/DevOps responsibility	Flutter responsibility
S3	Create bucket policy, upload URL, object lifecycle, access rules	Use presigned URL; upload bytes only
SQS	Queue and consume processing jobs	Never call SQS directly
RDS PostgreSQL + PostGIS	Schema, migrations, indexes, spatial queries	No direct DB access
ECS/Fargate	Run API/worker containers	None
ECR	Store images	None
CloudWatch	Logs, metrics, alarms	Surface request/job ID if useful
Secrets Manager/SSM	Store production secrets	None

21. Recommended Repository Split

```

rodesence/
  backend/    # Fastify API + DB + AWS orchestration
  ml-worker/  # Python + FFmpeg/OpenCV + YOLO/equivalent
  frontend/   # Flutter mobile/web app
  infrastructure/ # Terraform or AWS IaC (after MVP works)
  docs/       # API contract, architecture, model notes

```

If the team wants separate Git repositories, keep the same logical boundaries. The important point is that the ML worker and Fastify API are independent deployment units, even if the initial implementation shares a repository.

22. Definition of Done - Integration

- Flutter can authenticate and receive documented credentials.
- Flutter can request a presigned S3 upload URL and upload a test video.
- Flutter can start processing and receive a processing job ID.
- Flutter can display processing state after app restart.
- An ML worker can consume a processing job without accessing Flutter code.
- At least one detection record can be written with bounding box + confidence.
- A severity result is persisted with a version and explanatory factors.
- Frame timestamps can be synchronized to positional data for at least the provided dataset format.
- Flutter can load a viewport of detections and render them on a map.
- Frontend can filter detections by severity/category/date.
- Admin can see priority-ranked detections/issues.
- All protected operations reject unauthorized users consistently.
- API errors follow the documented error contract.
- All required environment variables are documented in .env.example.

23. Hackathon Priority Order

Priority	Build
P0 - must work	Video upload -> S3 -> processing job -> frame extraction -> YOLO/equivalent detection -> confidence/bbox -> severity -> GPS

	sync -> map
P1 - strong differentiator	Priority score with explainable factors; detection clustering/deduplication; map filters; processing progress
P2 - product layer	Citizen reports; worker assignment; admin verification; notifications
P3 - post-hackathon	Advanced routing optimization, real-time worker tracking, sophisticated analytics, extensive automation

24. Source and Scope Note

Source basis: PARAKRAM 1.0, GITA Autonomous College, Problem Statement ID PK01PS001, “RoadSense: Spotting Trouble Before It Spreads - Mapping the Damage the Naked Eye Overlooks.” The official brief states the challenge requirements and judging criteria on pages 1-3. This document combines those requirements with the team's current architecture decisions; items labelled as extensions (such as citizen reports and worker assignment) are product additions, not requirements explicitly mandated by the brief.

Contract owner: Rodesence backend + frontend leads. Any API field, endpoint, state name, enum, or authorization rule that changes must be updated here before the frontend and AI implementation diverge.